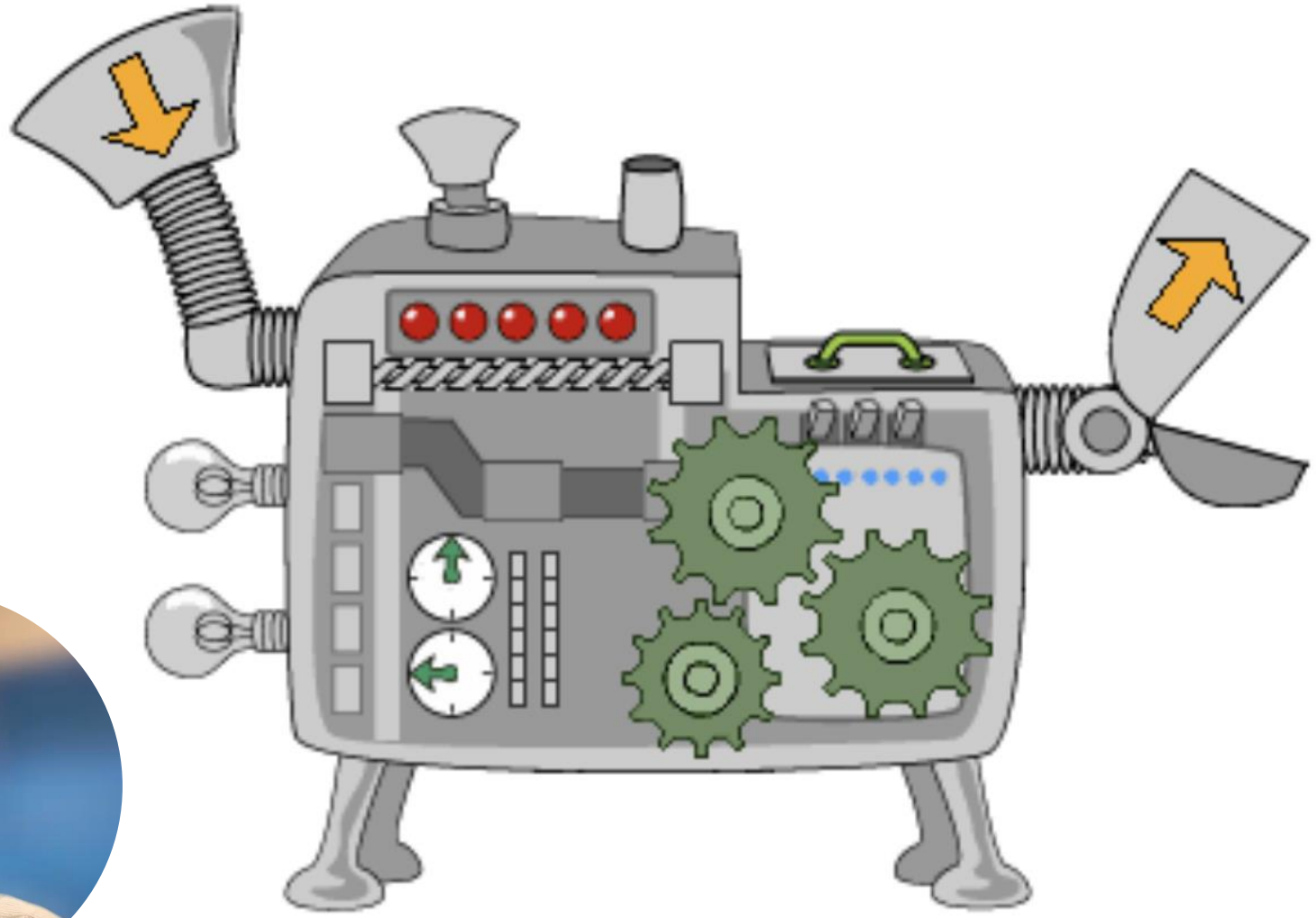
University of Exeter

ECM2433: The C Family

Files and streams

Overview

- Console input
- Streams and Files
  - file I/O
  - standard streams



https://www.iloveaba.com/2016/11/input-output.html

Console input

scanf() is like the reverse of printf(): Formatted input

```
#include <stdio.h>

int main()
{
    int k;

    do {
        printf("Please enter an integer: ");
        if (scanf("%d", &k) != 1)
            break;
        printf("Read %d\n", k);
    } while (1);
    return 0;
}
```

returns the number of data items successfully assigned a value

Program output

A **stream**

- is an abstraction of a **file**
  - provides a consistent interface to the programmer, regardless of the actual device
- is **buffered**
  - data is not necessarily written to the device when the write command is issued
  - you can force the write to happen by **flush**ing a stream
- may be **text** or **binary**
  - text stream = sequence of characters
    character translation may occur, e.g. newline → carriage-return + line-feed
  - binary stream = sequence of bytes
    no character translation occurs
- need to #include **stdio.h**

Using a stream

```
#include <stdio.h>
#include <stdlib.h>

int main ()
{
   FILE *fp;      /* file pointer */

   fp = fopen("myTextFile.lis", "r");

   if (fp == NULL) {
      printf("Cannot open file for read access\n");
      exit(EXIT_FAILURE);
   }

   /* do some file processing */

   fclose(fp);
   return EXIT_SUCCESS;
}
```

Prototype:

FILE *fopen(const char *path, const char *mode);

Opening a file: file modes

mode	meaning
r	open a text file for reading from
w	create a text file for writing to
a	open a text file for appending to (writing to the end of it)
rb	open a binary file for reading from
wb	create a binary file for writing to
ab	open a binary file for appending to (writing to the end of it)
r+	open a text file for reading from and writing to
w+	create a text file for reading from and writing to
a+	open a text file for reading from and appending to
r+b	open a binary file for reading from and writing to
w+b	create a binary file for reading from and writing to
a+b	open a binary file for reading from and appending to

Common ANSI C stream functions

University of Exeter

functions	action
fopen()	open a stream; must do this before you can read from or write to the stream
fclose()	close a stream; must do this before ending the program
putc() or fputc()	write a character to a stream
getc() or fgetc()	read a character from a stream
fputs()	write a string to a stream
fgets()	read a string from a stream
fprintf()	like printf(), but writing to a stream instead of the screen
fscanf()	like scanf(), but reading from a stream instead of the console
feof()	returns true if end of stream reached
ferror()	returns true if an error has occurred when using a stream
fflush()	forces the output buffer to be written to the stream
fseek()	jump to a specific byte in a stream
rewind()	resets the file position (for read/write) back to the brginning of the stre
remove()	delete a file
fwrite()	write to a binary stream
fread()	read from a binary stream

Using a stream

Always test that opening the file was successful:

```
fp = fopen("myTextFile.lis", "r");
if ( fp == NULL )
{
   /* do some error processing */
}
```

usually compressed into:

```
if ( (fp = fopen("myTextFile.lis", "r")) == NULL )
{
   /* do some error processing */
}
```

Trapping errors: part 1

University of Exeter

`errno.h` defines `int errno` (A global variable!)

- value is updated by system functions when an error occurs
- you can reset the value

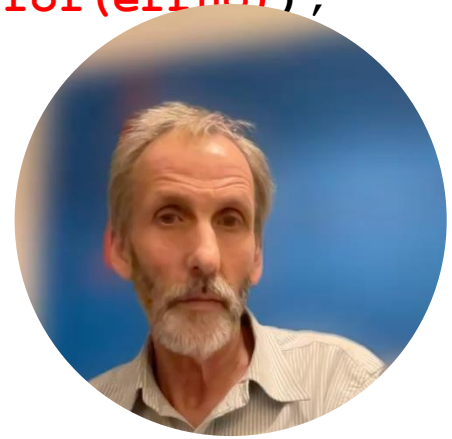
"The initial value of `errno` at program startup is zero. Many library functions are guaranteed to set it to certain nonzero values when they encounter certain kinds of errors. These error conditions are listed for each function. These functions do not change `errno` when they succeed; thus, the value of `errno` after a successful call is not necessarily zero, and you should not use `errno` to determine whether a call failed. The proper way to do that is documented for each function. If the call failed, you can examine `errno`."

GNU manual,

Trapping errors: part 1

Example code:

```
#include <errno.h>
#include <string.h>
#include <stdio.h>

int main(int argc, char **argv)
{
    FILE *fp;

    if (argc != 2) {
        printf("Expected exactly one argument, got %d\n", argc-1);
        return -1;
    }

    if ( (fp = fopen(argv[1], "r")) == NULL ) {
        printf("Unable to read file %s: %d: %s\n", argv[1], errno, strerror(errno));
        return 1;
    }
    return 0;
}
```



```
Unable to read file foo: 2: No such file or directory
```

Trapping errors: part 1

University of Exeter

Malloc

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>

int main ()
{
    int *p = NULL;

    if (!(p = (int *)malloc(100000000000000*sizeof(int) ))) {
        printf("Malloc failed. %d: %s\n", errno, strerror(errno));
        return -1;
    }

    /* Do something with p */
    return 0;
}
```

```
Malloc failed. 12: Cannot allocate memory
```

Trapping errors: part 2

A call to `ferror(fp)`

- determines whether the most recent operation on the file pointed to by `fp` produced an error
- returns 1 if an error occurred; 0 if not
- value is reset by each file operation

Write to a stream

- `fputc(ch,fp)`
  - outputs the character in `ch` to the file pointed to by `fp`

- `fputs(str,fp)`
  - outputs the string in `str` to the file pointed to by `fp`
  - the null terminator is not output

- `fprintf(fp, "Character %c, integer %03d", ch, i);`
  - like `printf`, but the string is output to the file pointed to by `fp`

Read from a stream

University of Exeter

- `char ch = fgetc(fp);`
  - reads a single character from the file pointed to by `fp` and stores it in `ch`

- `fgets(str,count,fp);`
  - reads multiple characters up to and including the next newline character (or a maximum of `count` characters) from the file pointed to by `fp` and stores it in the string `str`
  - adds a null terminator to `str`
  - cannot distinguish between null character read in and the terminating null character

- `fscanf(fp,"%d,%d",&x,&y);`
  - like `scanf`, but the string is input from the file pointed to by `fp`

Other common stream functions

- `fflush(FILE *fp);`
  - forces the output buffer to be written to the file
- `remove("myFile.txt");`
  - deletes the specified file
  - returns 0 on success; other integer on failure
- `rewind(FILE *fp);`
  - resets the file position indicator back to the beginning of the file
- `int fseek(FILE *stream, long offset, int whence);`
  - move the file position indicator to `offset` bytes from `whence` (beginning, end or current position)

Standard streams

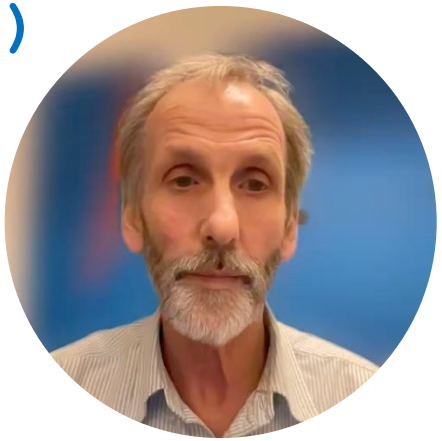
There are three standard streams that all programs can use **without** calling `fopen` or `fclose`:

- `stdin` (standard input, input from the terminal
- `stdout` (standard output) output to the terminal; normal program output
- `stderr` (standard error) output to the terminal; error and warning messages

unless redirected

`scanf` reads input from `stdin`; equivalent to `fscanf(stdin,...)`

`printf` sends output to `stdout`; equivalent to `fprintf(stdout,...)`



Write normal output to stdout; errors to stderr

```
#include <stdlib.h>
#include <stdio.h>
#include <errno.h>
#include <string.h>

int main(int argc, char **argv)
{
    FILE *fp;
    char  ch;

    printf("Hello from %s\n", argv[0]);

    if ( (fp = fopen(argv[1], "r")) == NULL ) {
        fprintf(stderr, "Unable to open file for read access.\n");
        fprintf(stderr, "error %d: %s\n", errno, strerror(errno));
        exit(1);
    }

    while ( !feof(fp) ) {
        ch = fgetc(fp);
        printf("%3d: %c\n",ch, ch);
    }

    fclose(fp);
    return EXIT_SUCCESS;
}
```

stdout vs stderr

When your program is run from the Linux command line it is likely, in a real system, to be run like this ($ is the Linux prompt):

```
$ file-read-fgetc nosuchfile > output 2> errors
```

The first redirection (>) sends the **stdout** output to a file called output

The second redirection (2>) sends the **stderr** output to a file called errors.

```
$ cat output
Hello from file-read-fget
$ cat errors
Unable to open "nosuchfile" for read access.
error 2: No such file or directory
```

To send both streams to the same output file:

```
$ myprogram > out.lis 2>&
```

Streams are buffered

University of Exeter

```
#include <stdio.h>
#include <string.h>

int main ()
{
   char   str[20] = "";
   char  *p = NULL;

   printf("Step 1, ");
   strcpy(str, "Hello World!");
   printf("Step 2, ");
   /* The following line should generate a memory fault */
   strcpy(p, "Hello World!");
   printf("Step 3\n");

   return 0;
}
```

What happens to the output produced by printf?

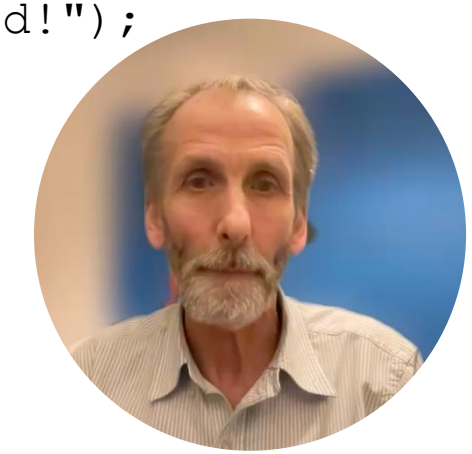
```
Memory fault(coredump)
```

Streams are buffered

```
#include <stdio.h>
#include <string.h>

int main ()
{
   char   str[20] = "";
   char  *p = NULL;

   printf("Step 1, ");
   strcpy(str,"Hello World!");
   printf("Step 2, ");
   fflush(stdout);
   strcpy(p,"Hello World!");
   printf("Step 3\n");

   return 0;
}
```

Flush the buffer of any pending output

```
Step 1, Step 2, Memory fault(coredump)
```



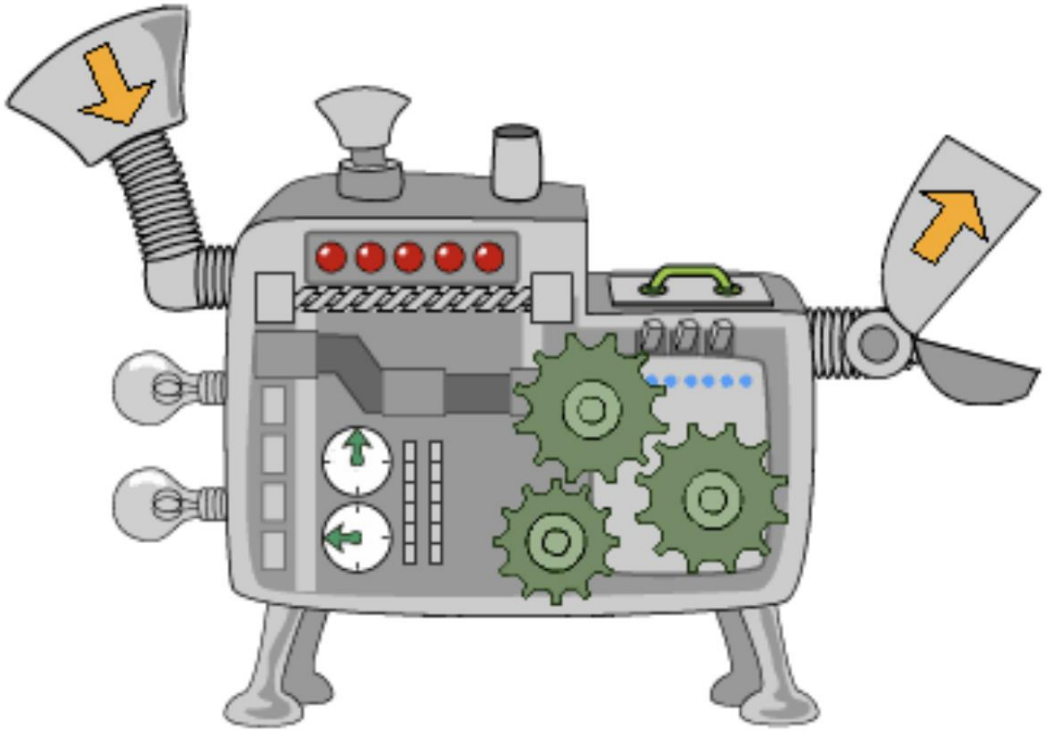
Streams are flushed:

- at end of each line ('\n') when output to a terminal
- when the buffer is full

- Buffer size is implementation dependent, but often 8192 bytes; can be altered with setbuf()
- 'Raw' (unbuffered) read and write can be achieved with read() and write()

Summary

- Streams are abstractions of files – “everything in *nix is a file”
- Errors tracked by (global) `errno`
- Standard input (stdin), standard output (stdout) and standard error (error) are always provided
- I/O is error prone – always check the result of an operation



https://www.iloveaba.com/2016/11/input-output.html